1. Title Page

Course Name: PCCCS495 – Term II Project
Project Title: Student Course Registration System

Student Name: Riddhi Banerjee

Roll Number: 69

Instructor Name: Soumadip Biswas

Spring 2026

2. Problem Statement

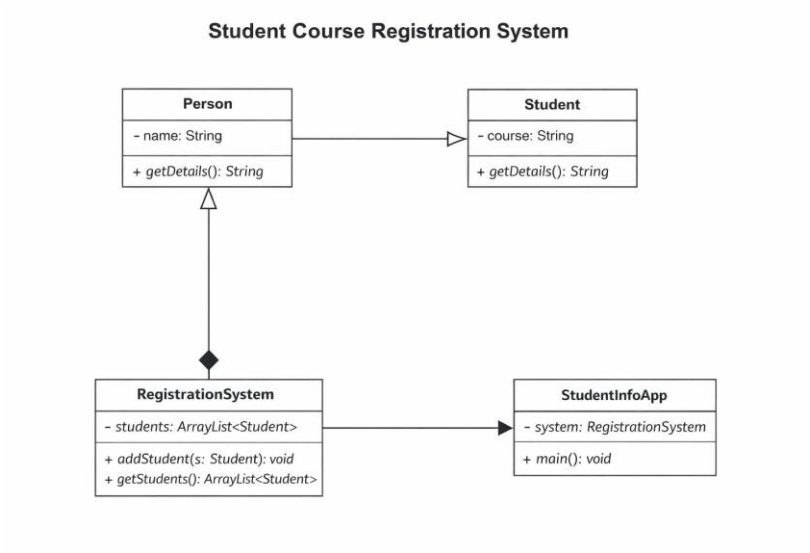
Managing student course registrations manually often leads to inefficiencies such as data duplication, human errors, and difficulty in maintaining records. Educational institutions require a simple and reliable system to handle student registration processes effectively.

This project proposes a Java-based GUI application that allows users to register students for selected courses. The system ensures proper input validation, prevents duplicate registrations, and displays student data dynamically. By integrating object-oriented programming concepts with a graphical interface, the application provides a structured and user-friendly solution for managing student-course information efficiently.

---

3. System Design

3.1 UML Class Diagram



3.2 Description of Major Classes

- **Person:**
  A base class that stores common attributes such as the student name.
- **Student:**
  Inherits from the Person class and includes additional attributes such as course details.
- **RegistrationSystem:**
  Manages student records using a collection and provides methods to add and retrieve student data.

- **StudentInfoApp:**
  The main GUI class that handles user interaction and connects the interface with backend logic.

3.3 Package Structure

```
src/
 ├── Person.java
 ├── Student.java
 ├── RegistrationSystem.java
 └── StudentInfoApp.java
```

3.4 Interaction Flow

1. User enters student name
2. User selects a course
3. Save button is clicked
4. Input validation is performed
5. Student object is created
6. Data is stored in the system
7. Display area is updated
8. Duplicate entries are checked and prevented

---

4. OOP Concepts Demonstrated

4.1 Abstraction

Where used: RegistrationSystem class
Why used: To separate data handling logic from GUI

```
public void addStudent(Student s) {
   students.add(s);
}
```

Explanation:
The method hides internal storage details and provides a simple interface to add students.

4.2 Inheritance

Where used: Student extends Person
Why used: To reuse common attributes

public class Student extends Person

Explanation:
Student inherits the name attribute from Person, reducing redundancy.

4.3 Polymorphism

Where used: Method overriding

```
@Override
public String getDetails() {
   return "Name: " + name + ", Course: " + course;
}
```

Explanation:
The method provides a specific implementation for Student objects.

4.4 Encapsulation

Where used: Private variables in classes
Why used: To protect data

private String course;

Explanation:
Restricts direct access and ensures controlled modification of data.

4.5 Collections

Where used: ArrayList

private ArrayList<Student> students = new ArrayList<>();

Explanation:
Allows dynamic storage of multiple student objects.

---

5. Implementation Highlights

Snippet 1: Input Validation

```
if (name.isEmpty()) {
   JOptionPane.showMessageDialog(frame, "Name cannot be empty!");
   return;
}
```

Justification:
Ensures that invalid input is not accepted, improving system reliability.

Snippet 2: Object Creation

```
Student student = new Student(name, course);
```

Justification:
Represents real-world entities using object-oriented design.

Snippet 3: Data Storage

```
system.addStudent(student);
```

Justification:
Uses abstraction to manage data through a dedicated class.

Snippet 4: Display Logic

```
for (Student s : system.getStudents()) {
   displayArea.append(s.getDetails() + "\n");
}
```

Justification:
Provides dynamic and updated display of stored data.

---

6. Testing & Error Handling

Test Cases Considered

- Valid input → Data stored correctly
- Empty input → Error message displayed
- Duplicate input → Entry blocked

Edge Cases Handled

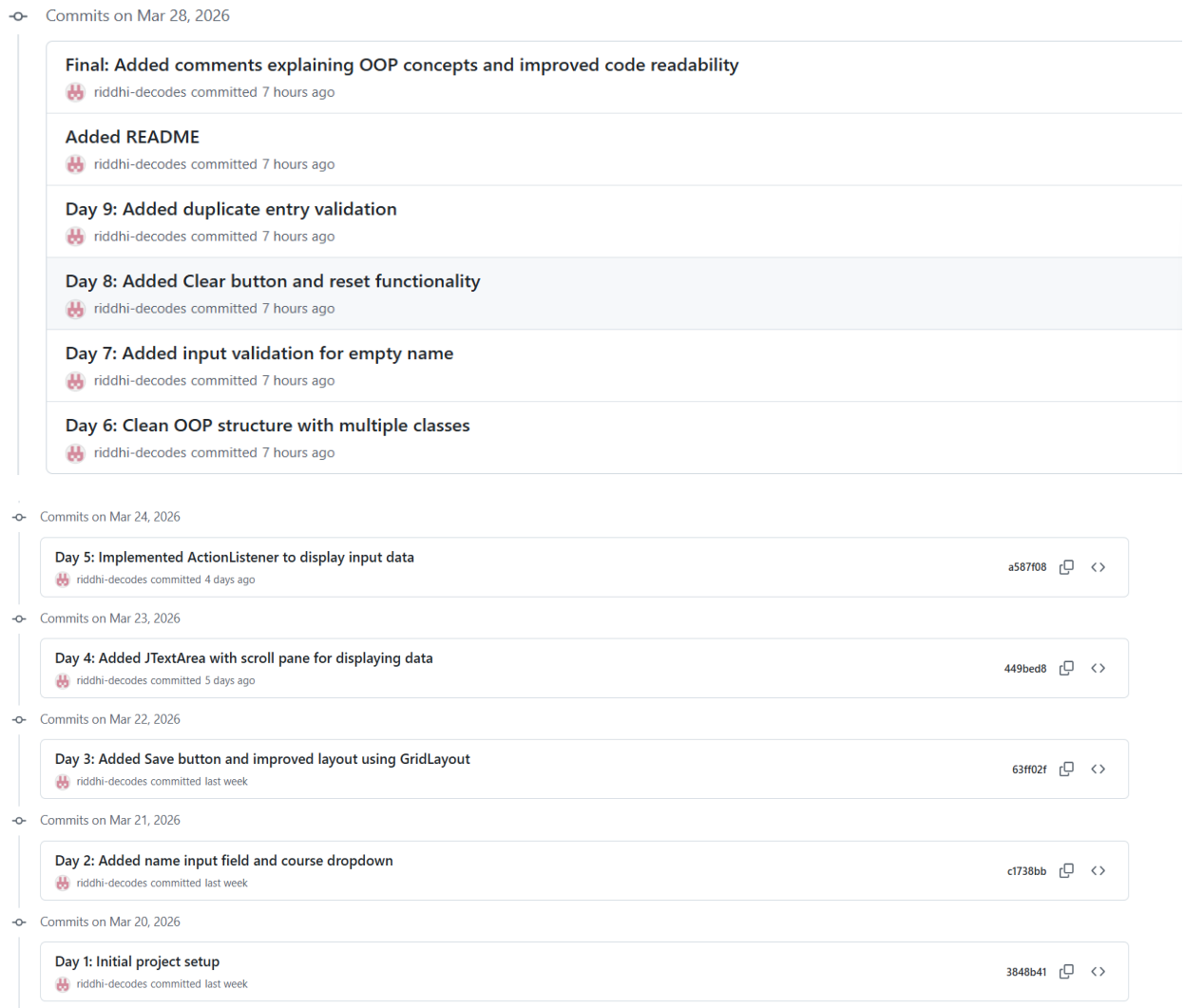
- Blank student name
- Repeated student-course combination

Failure Scenarios

- Invalid input handled using popup messages
- System prevents crashes and maintains stability

---

7. Git Workflow



Explanation:
The project was developed incrementally with meaningful commits. Each stage of development, including GUI setup, event handling, OOP implementation, validation, and documentation, was committed separately. This ensured proper version control and traceability of progress.

8. Conclusion & Future Scope

The project successfully demonstrates the integration of Java Swing with object-oriented programming concepts. It provides a simple yet effective solution for managing student course registrations while maintaining data accuracy and usability.

Future Scope:

- Integration with database for permanent storage
- Addition of search and delete features
- Improved GUI design
- Support for multiple course selection

Appendix:

Acceptance Mail: